Les 9 presentacions de classe — pinzellades

Conceptes clau de cada tema presentat a classe, per a la pregunta del control.

Els colors de la llista agrupen els temes en tres famílies:

Família	Temes
🩷 **Al client**	HTML5 · CSS & CSS Frameworks · JavaScript & DOM
💛 **Formats i estils d'API**	JSON & JSON Schema · SOAP & WSDL · OpenAPI & AsyncAPI · GraphQL
🩵 **Protocols**	WebSockets & Server-Sent Events · HTTP/2 & HTTP/3

🩷 1. HTML5

Què és: no una versió, sinó un **living standard** del WHATWG que s'actualitza contínuament. Defineix **estructura, semàntica i un conjunt d'APIs** — mai aparença.

El nucli:

- **Elements semàntics**: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`. Diuen **què és** cada tros, no com es veu. Importa per a **accessibilitat** i per als cercadors.
- **Formularis**: nous `type` (`email`, `date`, `number`, `range`, `color`, `tel`, `url`) i atributs de validació (`required`, `pattern`, `min`, `max`).
- **Multimèdia natiu**: `<audio>` i `<video>` sense connectors — és el que va matar Flash.
- **Gràfics**: `<canvas>` (ràster, el pintes amb JS) i **SVG inline** (vectorial, part del DOM).
- **Les APIs que va portar**: Web Storage (`localStorage` / `sessionStorage`), IndexedDB, Geolocation, Web Workers, WebSockets, File API, Drag & Drop i la **History API** (`pushState`).

Si et pregunten una cosa: que **la History API és el que fa possibles les SPA** — permet canviar la URL sense recarregar la pàgina, i per tant que el *routing* visqui al client.

I la trampa de sempre: la validació de formulari d'HTML5 és **comoditat per a l'usuari, mai seguretat**. Qualsevol se la salta amb les DevTools o amb `curl`. **Tota validació s'ha de repetir al servidor.**

🩷 2. CSS & CSS Frameworks

Què és: el llenguatge de presentació. La «C» és de **Cascading**: quan diverses regles afecten el mateix element, guanya segons **especificitat**, **origen** i **ordre**.

El nucli:

- **Box model**: content → padding → border → margin. I `box-sizing: border-box`, que fa que `width` inclogui padding i vora (el que tothom vol).
- **Evolució del layout**: taules → *floats* → **Flexbox** → **Grid**.
  - **Flexbox = una dimensió** (una fila *o* una columna).

    - **Grid = dues dimensions** (files *i* columnes alhora).
- **Responsive**: *media queries*, enfocament **mobile-first**, unitats relatives (`rem`, `em`, `%`, `vw`).
- **Preprocessadors** (Sass, Less): variables, *nesting*, *mixins*. Avui en bona part substituïts per les **custom properties** natives (`--color: red;` + `var(--color)`), que a més són dinàmiques en temps d'execució.
- **Frameworks**, i la distinció que val la pena saber:
    - **Bootstrap = basat en components**: et dona peces ja fetes (navbar, card, modal) amb el seu aspecte.
    - **Tailwind = *utility-first***: et dona classes atòmiques (`flex`, `pt-4`, `text-center`) i tu compones l'aspecte des de l'HTML.

Si et pregunten una cosa: Flexbox vs Grid (una dimensió vs dues), o **components vs utilitats**.

El compromís dels frameworks: guanyes velocitat i consistència; pagues mida del bundle, aspecte genèric, i acoblament del marcatge al framework.

🩷 3. JavaScript & DOM

Què és: el llenguatge que dona comportament a la pàgina, i el **DOM** és l'API que li permet tocar-la.

El nucli:

- **El DOM** és un **arbre de nodes** que el navegador construeix a partir de l'HTML. No és l'HTML: és la seva representació viva en memòria. `querySelector`, `createElement`, `addEventListener`.
- **Esdeveniments**: **capturing** (de fora cap endins) → **target** → **bubbling** (de dins cap enfora). El *bubbling* permet la **delegació**: un sol listener al pare en comptes de mil als fills.
- **Un sol fil + event loop.** Aquest és el concepte estrella. JavaScript té **un sol fil d'execució**: pila de crides, cua de tasques i microtasques. Per això un càlcul llarg **congela la pàgina**, i per això existeixen els **Web Workers**.
- **Asincronia, en tres generacions**: *callbacks* (i el *callback hell*) → **Promises** → **`async` / `await`**.
- **Rendiment**: tocar el DOM provoca **reflow i repaint**, que són cars. Per això convé agrupar els canvis — i per això van néixer els frameworks amb *virtual DOM*.
- **Mòduls** ES (`import` / `export`) i *bundlers*.

Si et pregunten una cosa: l'event loop — com pot ser que JavaScript sigui d'un sol fil i alhora no bloquejant. Resposta: les operacions d'E/S les fa el navegador fora del fil, i els resultats tornen a la cua; el fil només executa el que li toca de la cua, un a un.

💛 4. JSON & JSON Schema

Què és: el format d'intercanvi de dades del web modern. Neix com a subconjunt de la sintaxi literal de JavaScript.

El nucli:

- **Sis tipus i prou**: `string`, `number`, `boolean`, `null`, `object`, `array`. **No té dates, ni comentaris, ni comes finals.**
- **Per què va guanyar a XML**: més lleuger, **natiu al navegador** (`JSON.parse`, `JSON.stringify`), i molta menys cerimònia.
- **Què li falta respecte d'XML**: no porta esquema de sèrie, no té *namespaces*, no distingeix atributs d'elements, no té comentaris.
- **JSON Schema** omple aquell buit: és **un document JSON que descriu com han de ser altres documents JSON**. Paraules clau: `type`, `properties`, `required`, `enum`, `minimum`/`maximum`, `pattern`, `$ref`.
- Serveix per a **validar**, **documentar** i **generar codi** — i és **el sistema de tipus que fa servir OpenAPI** per dins.

Si et pregunten una cosa: JSON vs XML (lleugeresa i suport natiu contra esquemes, namespaces i eines), o que **JSON Schema és ell mateix JSON** — a diferència de WSDL, que és XML descrivint XML.

💛 5. SOAP & WSDL

Què és: l'estil de serveis web **anterior** a REST. SOAP és un **protocol** (no un estil arquitectònic), basat en XML.

El nucli:

- **Sobre XML**: `Envelope` que conté un `Header` (metadades: seguretat, transaccions) i un `Body` (la crida o la resposta).
- **Independent del transport**: normalment viatja per `POST` d'HTTP, però pot anar per SMTP o JMS.
- **Un sol endpoint, i les operacions van dins del cos.** Aquest és el contrast gran amb REST, que identifica **recursos per URI** i hi actua amb els **mètodes d'HTTP**.
- **WSDL** és el **contracte llegible per màquina**: `types`, `message`, `portType` (les operacions), `binding` (com s'envien) i `service` (on són). Permet **generar clients i stubs automàticament**.
- **L'stack WS-***: `WS-Security`, `WS-ReliableMessaging`, `WS-AtomicTransaction` — funcionalitats d'empresa que REST no té de sèrie.

Per què va caure: verbós, eines pesades, cost de parsejar XML, i no encaixa gens amb el navegador.

On sobreviu: banca, telecomunicacions, administració pública, integració entre sistemes d'empresa.

Si et pregunten una cosa: SOAP vs REST. SOAP és **orientat a operacions i contracte primer**; REST és **orientat a recursos i a la interfície uniforme d'HTTP**.

💛 6. OpenAPI & AsyncAPI

Què és: els contractes llegibles per màquina de l'era REST i de l'era dels esdeveniments. **OpenAPI és, conceptualment, el WSDL de REST.**

OpenAPI (abans Swagger):

- Document **YAML o JSON** que descriu `paths`, mètodes, paràmetres, els esquemes de petició i resposta (**amb JSON Schema**), els codis d'estat i l'autenticació.
- Permet **documentació interactiva** (Swagger UI), **generació de codi** de client i de servidor, **mocks** i **proves de contracte**.
- **Design-first vs code-first**: escriure primer el contracte i generar el codi, o escriure el codi i generar el contracte a partir d'anotacions.

AsyncAPI:

- La mateixa idea però per a sistemes **dirigits per esdeveniments**. On OpenAPI té `paths`, AsyncAPI té `channels`, `messages` i `servers`.
- Cobreix Kafka, MQTT, AMQP, WebSockets.
- **Imita l'estructura d'OpenAPI a posta**, perquè qui en coneix un entengui l'altre.

Si et pregunten una cosa: la parella. **OpenAPI descriu APIs de petició-resposta; AsyncAPI descriu publicació-subscripció.** I que tots dos fan per a REST i per als esdeveniments el que **WSDL** feia per a SOAP.

💛 7. GraphQL

Què és: un **llenguatge de consulta i un runtime** per a APIs, creat per Meta i alliberat el 2015.

El nucli:

- **Un sol endpoint** (normalment `POST /graphql`). El **client especifica exactament quins camps vol**, i rep exactament això.
- Això mata les dues malalties de REST: **over-fetching** (et porta camps que no necessites) i **under-fetching** (has de fer tres crides per pintar una pantalla). — *És el mateix problema que resol el patró **BFF** del tema 3, però per una altra via.*
- **Schema** amb sistema de tipus propi (SDL) i tres tipus arrel: `Query` (llegir), `Mutation` (escriure), `Subscription` (rebre actualitzacions en temps real — normalment sobre WebSockets).
- **Resolvers**: funcions que diuen d'on surt cada camp.
- **Versionat**: no es fan `/v2`; s'**evoluciona l'esquema** i es marquen camps com a `@deprecated`.

Els costos, que és el que demostra que ho has entès:

- **La cache d'HTTP deixa de servir.** Com que tot va per un `POST` a una sola URL, els mecanismes de cache basats en URL (navegador, reverse proxy, CDN) no hi poden fer res. Cal cache a nivell d'aplicació.

- **El problema N+1**: una consulta de 50 publicacions amb el seu autor pot acabar fent 51 consultes a la base de dades. Es resol amb *batching* (DataLoader).
- **Consultes malicioses**: una consulta profunda i recursiva pot tombar el servidor. Calen **límits de profunditat i de cost**.

Si et pregunten una cosa: over-fetching i under-fetching, i **per què la cache d'HTTP deixa de funcionar**.

💙 8. WebSockets & Server-Sent Events

(La vostra. Tens el document sencer a `presentacio_websockets_sse.md` .)

Què és: els dos mecanismes estàndard perquè **el servidor pugui enviar dades sense que el client les demani** — cosa que HTTP sol no permet (*«HTTP is one-way»*, tema 2, slide 26).

	SSE	**WebSockets**
Sentit	Servidor → client	Bidireccional i simultani
Protocol	HTTP, sense res més	Propi, després del handshake `101`
Dades	Només text UTF-8	Text i binari
Reconnexió	**Automàtica**, amb `Last-Event-ID`	Te l'escrius tu
API	`EventSource`	`WebSocket`
Complexitat	Baixa	Alta

Els tres fets que es recorden:

- El handshake de WebSocket comença com una petició HTTP amb `Upgrade: websocket` i es respon amb `101 Switching Protocols`. **A partir d'aquí ja no hi ha HTTP.**
- **El streaming de tokens d'un model de llenguatge va amb SSE**, no amb WebSockets: un cop enviada la pregunta, només parla el servidor.
- **Tots dos trenquen que el servidor sigui sense estat** → enllaç directe amb el tema 3: servidors replicats, *session affinity*, i cal un bus de publicació/subscripció entre nodes.

La crítica de seguretat: el handshake de WebSocket **no el protegeix la Same-Origin Policy**. Cal validar la capçalera `Origin` o tens **Cross-Site WebSocket Hijacking**.

💙 9. HTTP/2 & HTTP/3

Què és: les dues revisions del protocol que arreglen els colls d'ampolla d'HTTP/1.1.

HTTP/2 (RFC 7540, 2015):

- **Binari**, no text.
- **Multiplexació**: molts *streams* concurrents sobre **una sola connexió TCP** → elimina el límit de ~6 connexions per host i el *head-of-line blocking* **de la capa d'aplicació**.
- **HPACK**: compressió de capçaleres.

- **Server push**: **va fracassar.** Chrome el va desactivar per defecte el 2022; l'usava menys de l'1,3% dels llocs i no donava guany clar. L'alternativa és `103 Early Hints`. **No era una alternativa a SSE ni a WebSockets**: servia per empènyer **recursos a la cache**, no missatges d'aplicació.

HTTP/3 (sobre **QUIC**, que va sobre **UDP**):

- **El motiu d'existir:** HTTP/2 no va resoldre el *head-of-line blocking*, el va **moure a la capa de transport**. TCP entrega en ordre, o sigui que **si es perd un paquet, TOTS els streams s'aturen** — encara que siguin independents. Amb molta pèrdua, HTTP/2 pot anar **pitjor** que HTTP/1.1 amb sis connexions.
- **QUIC té els streams dins del transport**: una cua de reassemblatge **per stream** en comptes d'una de sola. Una pèrdua només atura el seu stream.
- **Per què sobre UDP**: TCP viu al nucli del sistema operatiu i els *middleboxes* el tenen fossilitzat. UDP permet que QUIC visqui en espai d'usuari i pugui evolucionar.
- **TLS integrat** (no és una capa a sobre, forma part del protocol).
- **0-RTT**: enviar dades al primer paquet reaprofitant una sessió anterior. **Només segur per a operacions safe o idempotents**, perquè el paquet es pot **reproduir**.
- **Connection migration**: QUIC identifica la connexió amb un **connection ID**, no amb la quàdrupla IP/port. Canviar de Wi-Fi a 4G **no talla la connexió**.

Si et pregunten una cosa: el *head-of-line blocking* i **on viu a cada versió**. És la pregunta que separa qui ha entès el tema de qui n'ha llegit la llista de característiques.

Les comparacions que travessen diversos temes

Si la pregunta és sobre «les presentacions» en plural, és probable que demani alguna cosa que en lligui més d'una. Les tres que jo em prepararia:

Estils d'API: SOAP vs REST vs GraphQL

	SOAP	REST	GraphQL
Unitat	**Operacions**	**Recursos**	**Camps d'un graf**
Endpoints	Un	Un per recurs	**Un de sol**
Contracte	**WSDL** (obligatori)	OpenAPI (opcional)	**Schema** (obligatori)
Format	XML	Normalment JSON	JSON
Cache d'HTTP	No	**Sí, de sèrie**	**No**
Versionat	Noves versions del WSDL	`/v2`	Deprecació de camps

Contractes: WSDL vs OpenAPI vs AsyncAPI

Tots tres fan la mateixa feina — **descriure una API perquè una màquina la pugui llegir** — per a tres mons diferents: **SOAP**, **REST** i **esdeveniments**. Els tres habiliten documentació, generació de codi i proves de contracte.

Temps real: SSE vs WebSockets vs HTTP/2 push

Els dos primers **sí** que serveixen per enviar missatges d'aplicació del servidor al client. **El server push d'HTTP/2 no**: servia per a recursos de cache i, a més, està desactivat. Confondre'ls és l'error típic.

I el fil que ho cus tot

Les nou presentacions van **de fora cap a dins i de la pàgina cap a la dada**:

```
Com es pinta la pàgina        HTML5 · CSS · JavaScript/DOM
Com es representen les dades  JSON & JSON Schema
Com es parlen els sistemes    SOAP/WSDL → OpenAPI/AsyncAPI → GraphQL
Sobre què viatja tot plegat   WebSockets/SSE · HTTP/2 i HTTP/3
```